CI/CD Architecture Proposal

Monorepo Microservices on GKE Autopilot

Prepared by: Armando Arredondo Valle
Date: April 27, 2026
Version: 1.0

---

1. Executive Summary

This proposal outlines a CI/CD architecture for scaling from a single Digital Ocean App Platform service to 7–8 microservices on GKE Autopilot. The design prioritizes **single-operator simplicity**, **cost efficiency**, and **vendor portability** — while preserving the automated git-push-to-deploy workflow you currently have.

These recommendations are grounded in hands-on experience: I've designed and operated YAML-based staged deployment pipelines with automated validation gates, slot-based zero-downtime rollouts, and environment promotion strategies across production distributed systems. I work daily with CI/CD orchestration, Docker containerization, and Kubernetes-based deployments at enterprise scale — including building end-to-end observability and incident response workflows across cloud-native services.

Recommended stack: GitHub Actions with self-hosted runners (or Bitbucket Pipelines if using Bitbucket), Harbor container registry, GKE Autopilot with namespace-based environment and zone segmentation.

---

2. Proposed Architecture

2.1 CI/CD Platform: GitHub Actions + Self-Hosted Runners (Recommended)

From my experience building and operating staged release pipelines with YAML-based configuration, automated validation gates, and multi-environment promotion — GitHub Actions is the strongest fit for a single-operator monorepo setup. I've operated similar architectures with environment-gated deployments, automated rollback triggers, and parallel build matrices.

Why GitHub Actions over alternatives:

Factor	GitHub Actions	Bitbucket Pipelines	GoCD	Drone CI / Harness
Monorepo path filtering	✅ Native (`dorny/paths-filter`)	✅ Native (condition-based)	❌ Manual	⚠️ Limited
YAML readability	✅ Clean, well-documented	✅ Clean, good docs	N/A (visual)	❌ Obscure syntax
Long-term maintenance	✅ GitHub-maintained	✅ Atlassian-maintained	❌ Stalled	⚠️ Uncertain
Single-pane visibility	✅ Actions tab + dashboards	✅ Deployment dashboard	✅ Good	✅ Good
Cost (self-hosted runners)	✅ Free (unlimited)	⚠️ 50 min/mo free, then paid	✅ Free	✅ Free
Self-hosted runners	✅ Supported	✅ Supported	✅ Native	✅ Native
Community / ecosystem	✅ Largest marketplace	⚠️ Smaller, but Pipes work	❌ Small	⚠️ Growing
Docker layer caching	✅ `docker/build-push-action`	✅ Native layer caching	⚠️ Manual	✅ Good

Primary recommendation: GitHub Actions — since you're already on GitHub, this adds zero new tooling. Self-hosted runners on your spare VPCs = unlimited free minutes.

> ***Alternative: Bitbucket Pipelines*** *— If your team uses or plans to move to Bitbucket (or wants tighter Jira/Confluence integration), Bitbucket Pipelines is a strong alternative. It supports monorepo path-based triggers natively, has a clean deployment dashboard, and supports self-hosted runners to avoid the free-tier minute limit. The YAML syntax is slightly different but equally readable. I can provide pipeline configurations for either platform — the Kustomize, Harbor, and GKE deployment steps are identical regardless of CI/CD provider.*

Self-hosted runners deployed on your existing spare VPCs eliminate hosted runner costs and keep build artifacts within your network. A single runner VM (2 vCPU, 4GB) handles the expected workload; a second runner can be added later for parallelism.

2.2 Container Registry: Harbor (Self-Hosted, CNCF)

Harbor runs on your VPC alongside the runners. Benefits:

- **No vendor lock-in** — portable across any cloud
- **Built-in vulnerability scanning** (Trivy integration) — security gate before deployment
- **Role-based access control** — image pull scoped per environment
- **Image promotion** — tag-based promotion from `test` to `prod` without rebuilding

Alternative if simplicity is preferred short-term: GitHub Container Registry (ghcr.io) — zero setup, free for private repos, but introduces mild GitHub dependency.

2.3 GKE Autopilot: Environment & Zone Segmentation

```
┌──────────────────────────────────────────────────────┐
│                  GKE Autopilot Cluster               │
│                                                      │
│  ┌──────────────────────┐  ┌────────────────────────┐ │
│  │   EXTERNAL ZONE      │  │    INTERNAL ZONE       │ │
│  │                      │  │                        │ │
│  │  ┌──────────────┐    │  │  ┌──────────────────┐  │ │
│  │  │ API Backend  │────│──│─▶│ Gateway Service  │  │ │
│  │  │  (Node.js)   │    │  │  └──────────┬───────┘  │ │
│  │  └──────────────┘    │  │             │          │ │
│  │                      │  │     ┌───────┴──────┐   │ │
│  │  ┌──────────────┐    │  │     ▼              ▼   │ │
│  │  │ Web Frontend │    │  │  ┌──────┐  ┌──────┐    │ │
│  │  │  (Vue.js)    │    │  │  │ Svc 1│  │ Svc 2│    │ │
│  │  └──────────────┘    │  │  └──┬───┘  └──┬───┘    │ │
│  │                      │  │     │  Kafka  │        │ │
│  └──────────────────────┘  │  ┌──▼─────────▼──────┐  │ │
│                            │  │ Confluent Cloud   │  │ │
│                            │  │   (Kafka SaaS)    │  │ │
│                            │  └───────────────────┘  │ │
│                            └────────────────────────┘ │
└──────────────────────────────────────────────────────┘
```

Segmentation approach:

- **Namespaces:** `external-test`, `external-prod`, `internal-test`, `internal-prod`
- **NetworkPolicies:** Internal namespace denies all ingress except from Gateway Service
- **Cloud NAT:** Single egress IP for all GKE pods — solves the Digital Ocean $25/IP problem for MongoDB Atlas whitelisting
- **Environments per cluster:** Single cluster, namespace-separated (cost-efficient at current scale; split to separate clusters when team grows)

---

3. CI/CD Pipeline Flow

3.1 Monorepo Structure

```
repo/
├── services/
│   ├── api-backend/        # Public API (Node.js)
│   ├── web-frontend/       # Vue.js SPA
│   ├── gateway/            # Internal gateway
│   ├── service-orders/     # Event-driven microservice
│   ├── service-payments/   # Event-driven microservice
│   ├── service-notifications/
│   └── service-inventory/
├── k8s/
│   ├── base/               # Shared Kustomize base manifests
│   ├── overlays/
│   │   ├── test/           # Test env variable overrides
│   │   └── prod/           # Prod env variable overrides
├── tests/
│   ├── api/                # Postman/Newman collections
│   └── security/           # ZAP/Trivy scan configs
└── .github/                          # ── OR ──
    └── workflows/
        └── ci-cd.yml                 # GitHub Actions workflow
└── bitbucket-pipelines.yml           # Bitbucket Pipelines (if applicable)
```

3.2 Pipeline Stages

```
Developer pushes to main branch
         │
         ▼
┌──────────────────────────┐
│ 1. DETECT CHANGES        │  dorny/paths-filter identifies which
│    (Path Filtering)      │  service folder(s) changed
└──────────────────────────┘
         │
         ▼
┌──────────────────────────┐
│ 2. BUILD & SCAN          │  Docker build → Trivy vulnerability
│    (Per changed service)│  scan → Push to Harbor with SHA tag
└──────────────────────────┘
         │
         ▼
┌──────────────────────────┐
│ 3. DEPLOY TO TEST        │  kubectl apply via Kustomize overlay
│    (Auto, on push)       │  (test namespace, test env vars)
└──────────────────────────┘
         │
         ▼
┌──────────────────────────┐
│ 4. POST-DEPLOY TESTS     │  Newman API tests (contract + auth)
│    (Automated)           │  + OWASP ZAP baseline security scan
└──────────────────────────┘
         │
         ▼
┌──────────────────────────┐
│ 5. REPORT & GATE         │  GitHub Actions summary: build, scan,
│    (Pass/Fail)           │  test results in single dashboard
└──────────────────────────┘
         │
```

```
        ▼
    Manual testing by operator
        │
        ▼
┌──────────────────────────┐
│ 6. PROMOTE TO PROD       │  Manual trigger (workflow_dispatch)
│    (Same image, new env)│  Same SHA-tagged image → prod namespace
└──────────────────────────┘  with production env vars via Kustomize
```

Key principle: The exact Docker image validated in test is promoted to production. No rebuild. Only environment variables change via Kustomize overlays.

---

4. Testing Strategy

4.1 Automated Tests (Post-Deploy)

Test Type	Tool	Purpose
API contract tests	**Newman** (Postman CLI)	Validate endpoints return expected schemas
Authorization checks	**Newman**	Verify unauthenticated requests are rejected (401/403)
Security baseline	**OWASP ZAP** (baseline scan)	Detect common vulnerabilities (XSS, injection, headers)
Image vulnerabilities	**Trivy**	Block deployment if critical CVEs found in container

4.2 Test Templates Provided

- Newman collection template with auth token flow + environment variable injection
- ZAP baseline scan GitHub Actions step configuration
- Trivy scan-and-gate step (fail pipeline on CRITICAL/HIGH severity)

---

5. Scaling a New Microservice

Adding a new microservice requires **3 steps** (< 30 minutes):

1. **Create service folder** in `services/` with Dockerfile
2. **Add Kustomize manifests** in `k8s/base/` (copy existing service template, change name/port)
3. **Add path filter entry** in `ci-cd.yml` — one line: `service-new: 'services/service-new/**'`

The pipeline automatically picks it up on next push. No infrastructure changes needed — GKE Autopilot scales nodes automatically.

---

6. Cost Estimate (Monthly)

Component	Cost	Notes
GKE Autopilot	~$75–150	Pay-per-pod, scales to zero when idle
Cloud NAT	~$5	Single egress IP (solves DO $25/IP problem)
Harbor on VPC	$0 incremental	Runs on existing spare VPC capacity
Self-hosted runners	$0 incremental	Runs on existing spare VPC capacity
GitHub Actions	$0	Free with self-hosted runners
MongoDB Atlas	Existing	No change

Confluent Cloud	Existing	No change
Total incremental	**~$80–155/mo**	

Compared to Digital Ocean App Platform at scale: 8 services × $25 egress IP = $200/mo for IPs alone, plus per-app compute costs.

---

7. Long-Term Portability

Every component is chosen for **zero vendor lock-in:**

Component	Portable?	Migration path
CI/CD YAML (GH or BB)	✅	GitHub Actions ↔ Bitbucket Pipelines ↔ GitLab CI (same patterns)
Harbor registry	✅	Self-hosted, runs anywhere (Docker Compose or K8s)
Kustomize manifests	✅	Standard K8s — works on any cluster (EKS, AKS, on-prem)
GKE Autopilot	✅	Standard K8s APIs, no GKE-specific features used
Newman/ZAP tests	✅	CLI tools, cloud-agnostic

If you move to AWS (EKS) or Azure (AKS) in the future, the only change is the kubectl context and Cloud NAT equivalent — all pipeline configuration, manifests, and tests transfer unchanged. I've worked across Azure, AWS, and GCP-based infrastructure, so I can ensure the architecture stays portable from day one.

---

8. Deliverables & Next Steps

#	Deliverable	Description
1	✅ This proposal	Architecture design with tradeoff analysis
2	CI/CD pipeline config	Complete GitHub Actions `ci-cd.yml` **or** `bitbucket-pipelines.yml` (your choice) with path filtering, build, scan, deploy, test, and manual promotion
3	Kustomize templates	Base + overlay manifests for all services, both environments
4	Harbor setup guide	Docker Compose config for Harbor on existing VPC
5	Runner setup	Self-hosted runner installation script for VPC (GitHub or Bitbucket runner)
6	Test templates	Newman collection template + ZAP baseline + Trivy gate config
7	Network policies	K8s NetworkPolicy manifests for external/internal zone isolation

Recommended kickoff: 30-minute call to review this proposal, confirm service boundaries, align on monorepo folder structure, and decide on CI/CD platform (GitHub Actions vs Bitbucket Pipelines) before starting configuration work.